# Defensive Security Assessment

An authorized assessment of my own local application, NoteKeeper (a small Node.js notes app). I tested it, documented each finding so it can be reproduced, fixed the problems, and re-ran the same tests on the fixed version.

**Result:** 10 findings (1 critical, 5 high, 3 medium, 1 low). 10 of 10 fixed and verified. The same 10 automated checks were run before and after, plus a code review (F3), a dependency audit (F9) and a log check (F10).

---

## 1. Scope, authorization and rules of engagement

### Scope

- **In scope:** NoteKeeper v1 running at `127.0.0.1:3001` on my own machine. Its code, configuration, dependencies and logs.
- **Fixed copy:** NoteKeeper v2 at `127.0.0.1:3002`, used for verification.
- **Out of scope:** every other host, network, service or person.

### Authorization statement

I wrote and own NoteKeeper and the machine it runs on. I authorize this security assessment of it, performed by me on 13 October 2026, for learning and defensive purposes. No third-party system, account or data was touched.

### Rules of engagement

- Localhost only. The test script refuses any other address.
- Synthetic data only: three fake users with made-up demo passwords. No real credentials or personal data anywhere.

- Low volume: about 30 requests per run, no denial-of-service or scanning.
- Stop and investigate if the app behaves unexpectedly.
- Evidence is limited to status codes, headers and short messages. No session values are published.

Method: OWASP Top 10 and ASVS as a checklist, grouped into the five areas below. Severity reflects how easy the flaw is to exploit and how much damage it does.

## 2. What was tested

| Area | Checks performed | Findi |
|---|---|---|
| **Configuration** | Response headers, error handling, cookie attributes | F7, F8 |
| **Authentication** | Password storage, session IDs and expiry, logout, login errors, rate limiting, access control | F2, F3 F5 |
| **Input validation** | SQL injection in login and search, reflected XSS, malformed IDs | F1, F6 |
| **Dependencies** | `npm audit` on the pinned versions | F9 |
| **Logging** | Whether attacks leave a usable trace, and whether logs avoid secrets | F10 |

## 3. Findings summary

| ID | Finding | Area | Severity | Status |
|---|---|---|---|---|
| **F1** | SQL injection in login and search | Validation | Critical | Fixed ar verified |
| **F2** | Broken access control (IDOR) on notes | Authentication / | High | Fixed ar |

| | | | | |
|---|---|---|---|---|
| | | authorization | | verified |
| F3 | Passwords stored with unsalted MD5 | Authentication | High | Fixed ar verified |
| F4 | Weak session management | Authentication | High | Fixed ar verified |
| F6 | Reflected cross-site scripting (XSS) in search | Validation | High | Fixed ar verified |
| F9 | Vulnerable dependencies | Dependencies | High | Fixed ar verified |
| F5 | Login abuse: no rate limit and account enumeration | Authentication | Medium | Fixed ar verified |
| F8 | Verbose error messages leak internals | Configuration | Medium | Fixed ar verified |
| F10 | No security logging | Logging | Medium | Fixed ar verified |
| F7 | Missing security headers and server fingerprinting | Configuration | Low | Fixed ar verified |

## 4. Detailed findings

Each finding lists the cause, impact, steps to reproduce, evidence before and after, and the fix. Ordered by severity.

▼ **F1 · SQL injection in login and search** · Critical

**Area:** Validation

**What is wrong:** User input is pasted into SQL strings, so an attacker changes the query itself.

**Impact:** Log in as any user without knowing the password, and read other users' notes.

**How to reproduce (against v1, localhost only):**

```
POST /login  {"username":"alice' --","password":"wrong"}
GET  /search?q=x%' OR '1'='1' --   (logged in as alice)
```

**Before (v1)**

POST /login {"username":"alice' --","password":"wrong"} -> HTTP 200
GET /search?q=x%' OR '1'='1' -- as alice -> HTTP 200, other user's note RETURNED

**After the fix (v2)**

POST /login {"username":"alice' --","password":"wrong"} -> HTTP 401
GET /search?q=x%' OR '1'='1' -- as alice -> HTTP 200, other user's note not returned

**Fix:** All queries use parameters (`WHERE username = ?`). Inputs are checked for type and length before use.

### ▼ F2 · Broken access control (IDOR) on notes · High

**Area:** Authentication / authorization

**What is wrong:** `/notes/:id` loads a note by ID only and never checks who owns it.

**Impact:** Any logged-in user can read any other user's note by changing a number.

**How to reproduce (against v1, localhost only):**

```
Log in as alice, then GET /notes/2  (bob's note)
```

**Before (v1)**

alice requests GET /notes/2 (bob's note) -> HTTP 200

**After the fix (v2)**

alice requests GET /notes/2 (bob's note) -> HTTP 404

**Fix:** Query is `WHERE id = ? AND owner_id = ?`. Other users' notes return 404, and the attempt is logged. ID must be digits only.

### ▼ F3 · Passwords stored with unsalted MD5 · High

**Area:** Authentication

**What is wrong:** MD5 is fast and has no salt, so a leaked database is cracked in bulk and equal passwords look equal.

**Impact:** A database leak exposes most passwords quickly.

**How to reproduce (against v1, localhost only):**

```
Code review: v1 calls crypto.createHash('md5') for every password.
```

**Before (v1)**

v1: MD5 hash found in source

**After the fix (v2)**

v2: scrypt + timingSafeEqual found in source

**Fix:** Passwords use scrypt with a random 16-byte salt per user and a timing-safe comparison.

### ▼ F4 · Weak session management · High

**Area:** Authentication

**What is wrong:** Session IDs are 1001, 1002, 1003 (guessable), the cookie has no protection flags, sessions never expire, and there is no logout.

**Impact:** An attacker can guess a valid session ID and take over an account, or reuse a stolen cookie forever.

**How to reproduce (against v1, localhost only):**

```
Log in twice and compare the sid values; inspect Set-Cookie; POST /logout and reuse the old
```

**Before (v1)**

Cookie attributes: (no attributes); session ID is a small incrementing number
POST /logout -> HTTP 404; old cookie on GET /me -> HTTP 200

**After the fix (v2)**

Cookie attributes: HttpOnly; SameSite=Strict; Path=/; Max-Age=900; session ID is long and random
POST /logout -> HTTP 204; old cookie on GET /me -> HTTP 401

**Fix:** 256-bit random IDs, `HttpOnly; SameSite=Strict; Max-Age=900` (plus `Secure` in production), 15-minute idle and 8-hour absolute expiry on the server, and a real `POST /lo`

gout.

### ▼ F6 · Reflected cross-site scripting (XSS) in search · High

**Area:** Validation

**What is wrong:** The search term is placed into HTML without encoding.

**Impact:** A crafted link runs attacker JavaScript in the victim's logged-in session.

**How to reproduce (against v1, localhost only):**

```
GET /search?q=<script>alert(1)</script>  and check whether the tag comes back unescaped.
```

**Before (v1)**

payload reflected unescaped in response body

**After the fix (v2)**

payload reflected HTML-escaped in response body

**Fix:** HTML-escape all output, and send a strict Content-Security-Policy (`default-src 'none'`) as a second layer.

### ▼ F9 · Vulnerable dependencies · High

**Area:** Dependencies

**What is wrong:** Pinned old versions of express and lodash pull in packages with known advisories: body-parser (high), express (high), lodash (high), path-to-regexp (high), qs (high), cookie (low), send (low), serve-static (low).

**Impact:** Known, publicly documented flaws are present in the running code.

**How to reproduce (against v1, localhost only):**

```
Run npm audit in the v1 folder.
```

**Before (v1)**

v1: 8 advisories (5 high, 3 low)

**After the fix (v2)**

v2: 0 advisories

**Fix:** Upgraded to `express ^4.21.2` and removed lodash (not needed).

### ▼ F5 · Login abuse: no rate limit and account enumeration · Medium

**Area:** Authentication

**What is wrong:** Different messages reveal which usernames exist, and guessing is unlimited.

**Impact:** Attackers can confirm valid accounts and then guess passwords at full speed.

**How to reproduce (against v1, localhost only):**

```
Send a wrong password for a real and a fake user and compare messages; send 8 wrong passwor
```

**Before (v1)**

unknown user -> "User not found";
known user, wrong password ->
"Wrong password"
8 wrong passwords in a row for one account -> 401×8

**After the fix (v2)**

unknown user -> "Invalid username or password."; known user, wrong password -> "Invalid username or password."
8 wrong passwords in a row for one account -> 401×4, 429×4

**Fix:** One generic message, a dummy hash check for unknown users (same timing), and a 15-minute lock after 5 failures per IP and username.

### ▼ F8 · Verbose error messages leak internals · Medium

**Area:** Configuration

**What is wrong:** The error handler returns the SQL error and a full stack trace to the client.

**Impact:** Reveals table names, file paths and code structure that help the next attack.

**How to reproduce (against v1, localhost only):**

```
Log in, then GET /notes/abc
```

**Before (v1)**

GET /notes/abc -> HTTP 500, body contains SQL error and stack trace

**After the fix (v2)**

GET /notes/abc -> HTTP 400, generic error only

**Fix:** Generic client errors (`Invalid request.` or `Something went wrong.`); details stay in the server log only.

### ▼ F10 · No security logging · Medium

**Area:** Logging

**What is wrong:** v1 writes nothing about logins, failures or denied access.

**Impact:** An attack like F1 or F5 would leave no trace, so it could not be detected or investigated.

**How to reproduce (against v1, localhost only):**

```
Run the attack tests, then count log lines the server wrote.
```

**Before (v1)**

v1: 0 log lines after the test run

**After the fix (v2)**

v2: 7× login_failure, 2× login_success, 1× access_denied, 1× logout, 4× rate_limited

**Fix:** Structured JSON events: `login_success`, `login_failure`, `access_denied`, `rate_limited`, `logout`. No passwords or session IDs are logged, and JSON encoding stops log injection.

### ▼ F7 · Missing security headers and server fingerprinting · Low

**Area:** Configuration

**What is wrong:** The app announces `X-Powered-By: Express` and sends no browser protections.

**Impact:** Makes targeting easier and removes cheap browser-side defences.

**How to reproduce (against v1, localhost only):**

```
GET /health and read the response headers.
```

**Before (v1)**

**After the fix (v2)**

| | |
|---|---|
| X-Powered-By: Express, no CSP, no nosniff | CSP, nosniff present; X-Powered-By removed |

**Fix:** Removed X-Powered-By. Added CSP, `X-Content-Type-Options: nosniff`, `Referrer-Policy` and `Cache-Control: no-store`.

## 5. Verification: same tests, before and after

Run with `node test.js http://127.0.0.1:3001` (v1) and `:3002` (v2). Real results from this session.

| ID | Check | v1 (before) | v2 (after) |
|---|---|---|---|
| F1a | SQL injection: login bypass | VULNERABLE | PASS |
| F1b | SQL injection: data leak in search | VULNERABLE | PASS |
| F2 | Broken access control (IDOR) | VULNERABLE | PASS |
| F4a | Session cookie flags and ID quality | VULNERABLE | PASS |
| F4b | Logout invalidates session | VULNERABLE | PASS |
| F6 | Reflected XSS in search | VULNERABLE | PASS |
| F7 | Security headers and fingerprinting | VULNERABLE | PASS |
| F8 | Verbose error messages | VULNERABLE | PASS |
| F5a | Account enumeration via login errors | VULNERABLE | PASS |
| F5b | No rate limit on login | VULNERABLE | PASS |
| F3 | Password hashing (code review) | MD5 | scrypt |
| F9 | Dependency audit (npm audit) | 8 advisories | 0 advisories |
| F10 | Security events logged | 0 lines | 15 events |

F5b detail: in v2 the lock starts after the 5th failed attempt for that account, so the 8-attempt run ends with a mix of 401 and 429 responses. One failed attempt had already been counted by the enumeration check just before it.

### Sample of v2 security log (no passwords or session IDs)


```
{"ts":"2026-10-13T11:49:09.446Z","event":"login_failure","user":"alice' --","ip":"127.0.0.1"}
{"ts":"2026-10-13T11:49:09.508Z","event":"login_success","user":"alice","ip":"127.0.0.1"}
{"ts":"2026-10-13T11:49:09.526Z","event":"access_denied","userId":1,"noteId":2}
{"ts":"2026-10-13T11:49:09.635Z","event":"login_success","user":"alice","ip":"127.0.0.1"}
{"ts":"2026-10-13T11:49:09.643Z","event":"logout","userId":1}
{"ts":"2026-10-13T11:49:09.710Z","event":"login_failure","user":"nobody","ip":"127.0.0.1"}
{"ts":"2026-10-13T11:49:09.772Z","event":"login_failure","user":"carol","ip":"127.0.0.1"}
{"ts":"2026-10-13T11:49:09.834Z","event":"login_failure","user":"carol","ip":"127.0.0.1"}
```


---

## 6. Residual risks and next steps

- **HTTPS:** the `Secure` cookie flag is only added in production. Deploy behind TLS.
- **State in memory:** sessions, lockouts and the database reset on restart. Production needs a real database and a shared store such as Redis.
- **Lockout trade-off:** locking an account after failures can be abused to annoy a victim. Pair it with IP-level limits and MFA.
- **Not covered:** MFA, password reset, CSRF tokens (SameSite=Strict reduces the risk), file upload, and a full manual penetration test. About 30 requests are not a complete audit.
- **Dependencies:** `npm audit` only knows published advisories. Re-run it regularly and enable automated updates.
- **Logs:** the log is only written to the console. Ship it to central storage and alert on repeated `login_failure` and `access_denied` events.

---

## 7. Short explanation

I started by writing down what I was allowed to test and how, so the work stays safe and repeatable. Then I attacked the app the way a real attacker would: I tried to bypass the login,

read other people's data, hijack sessions, inject script, trigger errors and guess passwords. Ten problems showed up, and the most serious (SQL injection) let me log in without a password.

I fixed the root cause of each one instead of patching symptoms: parameterized queries, ownership checks, random sessions with proper cookie flags, hashed passwords with a salt, output escaping, generic errors, rate limiting, updated dependencies and security logging. Then I ran the exact same tests on the fixed version. All of them passed, the dependency audit went from 8 advisories to none, and attacks now leave a trace in the log. I also listed what is still not covered, because a clean test run does not mean the app is fully secure.

---

## Appendix

▶ **NoteKeeper v1 source (vulnerable, for the assessment)**

▶ **NoteKeeper v2 source (fixed)**

▶ **test.js (assessment and verification script)**

▶ **Raw test results (JSON)**

▶ **Dependency audit output**